

Aprimoramento de reconhecimento óptico (Artigo: Isaac Caraça)

Autor: Isaac Caraça

Este artigo apresenta o desenvolvimento e aperfeiçoamento de um sistema automatizado para extração de texto a partir de documentos digitalizados, empregando técnicas de Optical Character Recognition (OCR). O objetivo principal é aumentar a confiabilidade da leitura de textos provenientes de arquivos PDF com baixa qualidade de digitalização, presença de marcas d'água e distorções gráficas. A solução foi desenvolvida em Python, utilizando as bibliotecas Tesseract OCR, OpenCV e PyMuPDF (Poppler), com módulos complementares de correção ortográfica e semântica automática. São discutidos os desafios encontrados na digitalização de documentos administrativos e os ganhos obtidos com a aplicação de pré-processamento de imagem e otimização de parâmetros do Tesseract.

Introdução

O reconhecimento óptico de caracteres (OCR) é uma tecnologia essencial para converter imagens ou PDFs digitalizados em texto pesquisável. No entanto, a qualidade do resultado depende diretamente da nitidez da imagem, presença de ruídos, marca d'água e tipo de fonte usada no documento original.

Neste artigo, mostraremos como otimizar o processo de OCR com Python usando Tesseract OCR, Poppler e OpenCV, aplicando técnicas de pré-processamento de imagem que melhoram significativamente a precisão do texto reconhecido — principalmente em documentos com baixa qualidade de digitalização ou marcas d'água.

Problema Real:

Durante os testes iniciais com o Tesseract OCR (antes da automação em Python), alguns registros — como o de ID 21312 — apresentavam trechos ilegíveis e palavras desconfiguradas devido à baixa resolução, sombras e marca d'água sobre o texto.

Exemplo de resultado anterior sem pré-processamento:

DECRETO N° 19542

de 27 de agosto de 1996

Dispõe Sobre: "Define atri s de sub-unidades

MUNICIPAL DE

ê Economia e Planejamento

ento e gerenciamento do Município

E em sintonia com os mais modernos recursos

Percebe-se que o OCR não conseguiu interpretar corretamente as palavras ("atri s", "ento", "E em") nem manter a formatação dos parágrafos.

Esses erros eram recorrentes em documentos com:

1. Baixo contraste entre texto e fundo;
2. Marcas d'água transparentes;
3. Imagens digitalizadas tortas ou com manchas.

Método:

Para solucionar esses problemas, foi desenvolvido um pipeline completo em Python com três etapas principais:

1. Pré-processamento das imagens (para aumentar contraste e remover ruídos);
2. Extração de texto com o Tesseract OCR;
3. Correção automática de erros ortográficos e reconstrução de parágrafos.

```
def preprocess_image(img_path):  
    """Pré-processa imagem para OCR mais confiável"""  
    img = cv2.imread(img_path)
```

```

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = cv2.bilateralFilter(gray, 9, 75, 75)
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
    gray = clahe.apply(gray)
    thresh = cv2.adaptiveThreshold(
        gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY, 15, 8
    )

    processed_path = img_path.replace(".png", "_proc.png")
    cv2.imwrite(processed_path, thresh)

    return processed_path

```

Explicação:

```
img = cv2.imread(img_path)
```

- Lê a imagem original do PDF convertida em PNG.

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

- Converte para tons de cinza, reduzindo o impacto das cores do fundo e marca d'água.

```
gray = cv2.bilateralFilter(gray, 9, 75, 75)
```

- Aplica filtro bilateral, que remove ruídos sem borrar as bordas das letras — essencial para OCR.

```
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
```

- Cria o algoritmo CLAHE (Equalização de Histograma Adaptativa), que aumenta o contraste localmente.

```
gray = clahe.apply(gray)
```

- Aplica o CLAHE, realçando letras apagadas ou sobre fundo manchado.

```
thresh = cv2.adaptiveThreshold(...)
```

- Cria uma imagem binária (preto e branco) adaptando o limiar de contraste por região, o que elimina sombras e marcas d'água leves.

```
cv2.imwrite(processed_path, thresh)
```

- Salva a imagem pré-processada que será usada pelo OCR.

```
def extrair_texto_tesseract(img_path):  
    """Extrai texto da imagem usando Tesseract OCR"""  
    img_proc = preprocess_image(img_path)  
    pil_img = Image.open(img_proc)  
    texto_final = pytesseract.image_to_string(pil_img, lang='por',  
config='--oem 3 --psm 4')  
    os.remove(img_proc)  
  
    return texto_final
```

Explicação:

```
img_proc = preprocess_image(img_path)
```

- Passa a imagem pelo filtro e equalização antes do OCR.

```
pytesseract.image_to_string(..., lang='por')
```

- Extrai o texto em português, respeitando acentuação e regras linguísticas.

```
config='--oem 3 --psm 4'
```

- Define o modo OEM 3 (OCR Engine Mode: LSTM + Legacy) e PSM 4 (análise de texto em blocos), ideal para documentos com múltiplas colunas.

```
os.remove(img_proc)
```

- Exclui a imagem temporária processada após a leitura.

Mesmo com tudo isso implementado ainda sim vinha muito resultado com erros de ortografia nas palavras e também de forma semântica vinha de forma errada, como:

```
Divisadã Técnica de Georáficas  
Profissão eximias: Georáfo  
Sagrou-Guarulhos (SIGNO)
```

```
Geoprocessamento- dos dados
```

Por isso foi pensado em utilizar se dos recursos disponíveis para melhorar ainda mais a qualidade, no caso:

Método:

```
CACHE = {}

DICCIONARIO_URL = "https://www.ime.usp.br/~pf/dicios/br-utf8.txt"
print("♦ Baixando dicionário...")
r = requests.get(DICCIONARIO_URL)
DICCIONARIO_BASE = set(line.strip().lower() for line in
r.text.splitlines() if line.strip())

DICCIONARIO_TECNICO = {
    "siggeo", "sig", "erpotech", "geotiff", "cad/gis", "qgis",
    "geoprocessamento", "geodésia", "geotecnologias",
    "georreferenciamento",
    "cartografia", "geoespacial", "teledetecção", "altimetria",
    "hidrografia", "topografia", "geocodificação",
    "georreferenciadas", "avanços", "divisão", "instalações",
    "unidade", "Seção", "empresas", "aculturamento"
}

DICCIONARIO = DICCIONARIO_BASE.union(DICCIONARIO_TECNICO)
print(f"[DICCIONÁRIO] {len(DICCIONARIO):,} palavras carregadas (incluindo
técnicas).")

TERMOS_ESPECIAIS = {"SIGeo", "ERPTech", "GeoTIFF", "CAD/GIS", "QGIS"}
TERMOS_ESPECIAIS_SEM_ACENTO = set(unidecode(t.lower()) for t in
TERMOS_ESPECIAIS)
TERMOS_APRENDIDOS = set()

CORRECOES_FIXAS = {
    "Divisadão": "Divisão",
    "Sintema": "Sistema",
    "Municípios": "Município",
    "à": "a",
    "Sagrou-Guarulhos": "Sigeo-Guarulhos",
    "anos": "aos",
    "asas": "as",
    "Projetos": "Projeto",
    "socâmo": "Seção",
    "Socão": "Seção",
    "arca": "área",
    "laorafo": "geógrafo",
    "vVAass": "administrativas",
    "Socâmo": "Seção",
    "AA": "a",
    "Carlos-": "cartográfico",
    "toronja-": "topografia",
    "Torna-r": "formatos",
    "alunização": "alimentação",
```

```

    "alunização": "alimentação",
    "suplementavas": "suplementadas",
    "BEL.": "Bacharel",
    "cão": "çã",
    "especificas": "específicas",
    "experiência na arca": "experiência na área",
    "Geo-": "Geo",
    "grafico": "gráfico",
    "grã-r": "gráficos",
    "da'": "da",
    "profissão eximias": "profissão exigida",
    "650": "Geógrafo",
    "Gerir à": "Gerir a",
    "às": "as",
    "Signo": "SIGeo",
    "SRH/SUL": "SCM/GUL", # manter
    "projétos": "projetos",
    "Geociências": "Geociências", # manter
    "informação": "Informação",
    "Georreferenciadas": "Georreferenciadas",
    "Referência".lower(): "referência",
    "Divisadã Técnica": "Divisão Técnica",
    "Socãmo Técnica": "Seção Técnica",
    "Geoprocessamento-": "Geoprocessamento",
    "Carlos-gráfico": "cartográfico",
    "marcos": "marcos",
    "posterior migração": "posterior migração",
    "suplementavas": "suplementadas",
    "Georáficas": "Geográficas",
    "de-": "de ",
    "Grã-r": "Gráficos",
    "Quadra-da": "Qualidade",
    "qualidade, e |": "qualidade, e",
    "topo -": "topográficos",
    "geo-r": "geodésicos",
    "Geotecnologias": "Geotecnologias"
}

```

```

CORRECOES_FRASES = {
    "Denominação da' |": "Denominação da",
    "Nível Universitário.com especializa-": "Nível Universitário com especialização",
    "Engenheiro, Geólogo. Geo- grafo ou Arquiteto": "Engenheiro, Geólogo, Geógrafo ou Arquiteto",
    "Georreferenciadas de Guarulhos (SIGNO-Guarulhos)": "Georreferenciadas de Guarulhos (SIGeo-Guarulhos)",
    "adequada e atrelada.aos": "adequada e atrelada aos",
    "geoprocessamento;s": "geoprocessamento;",
    "empresas privadas ou publicas prestaras de serviços": "empresas privadas ou públicas prestadoras de serviços",
    "questões fronteiriços": "questões fronteiriças",
}

```

```

    "administrati-administrativas": "administrativas",
    "Bases doe dados gráficos": "bases de dados gráficos",
    "ficou e alfanuméricos": "gráficos e alfanuméricos",
    "cruzamento de da-r dos existentes": "cruzamento dos dados existentes",
    "a fim doe carregá-las": "a fim de carregá-las",
    "Profissão eximias": "Profissão exigida",
    "continuamente atualizadas": "continuamente atualizada",
    "Carlos- gráfico": "cartográfico",
    "rede básica de apoio geodésico;s": "rede básica de apoio geodésico;",
    "toronja- fia": "topografia",
    "posterior migração": "posterior migração",
    "formatos tos' específicos": "formatos específicos",
    "suplementavas se necessário": "suplementadas se necessário"
}

```

```

def normalizar_hifen_linhas(texto):
    """
    Junta palavras quebradas por hífen no final da linha, preservando
    indentação e bullets.
    """
    linhas = texto.splitlines()
    linhas_corrigidas = []
    i = 0
    while i < len(linhas):
        linha = linhas[i]
        if linha.rstrip().endswith('-') and i + 1 < len(linhas):
            prox = linhas[i + 1].lstrip()
            linha = linha.rstrip()[:-1] + prox
            i += 1
        linhas_corrigidas.append(linha)
        i += 1
    return '\n'.join(linhas_corrigidas)

```

```

CORRECOES_FIXAS_EXPANDIDO = {}
for k, v in CORRECOES_FIXAS.items():
    CORRECOES_FIXAS_EXPANDIDO[k] = v
    sem_acento = unidecode(k)
    if sem_acento != k:
        CORRECOES_FIXAS_EXPANDIDO[sem_acento] = v

```

```

CORRECOES_FIXAS = CORRECOES_FIXAS_EXPANDIDO

```

```

def distancia_levenshtein(a, b):
    if a == b: return 0
    if len(a) == 0: return len(b)
    if len(b) == 0: return len(a)
    matrix = [[0]*(len(b)+1) for _ in range(len(a)+1)]

```

```

    for i in range(len(a)+1): matrix[i][0] = i
    for j in range(len(b)+1): matrix[0][j] = j
    for i in range(1, len(a)+1):
        for j in range(1, len(b)+1):
            custo = 0 if a[i-1] == b[j-1] else 1
            matrix[i][j] = min(matrix[i-1][j]+1, matrix[i][j-1]+1,
matrix[i-1][j-1]+custo)
    return matrix[len(a)][len(b)]

```

```

def aplicar_acentos(original, corrigida):
    resultado = ""
    for o, c in zip(original, corrigida):
        if re.match(r'[áéíóúãõâêôàèìòùç]', o.lower()):
            resultado += o
        else:
            resultado += c
    resultado += corrigida[len(resultado):]
    return resultado

```

```

def corrigir_frases(texto):
    for errado, certo in CORRECOES_FRASES.items():
        if errado in texto:
            texto = texto.replace(errado, certo)
            print(f"♦ Frase corrigida: '{errado}' -> '{certo}'")
    return texto

```

```

def corrigir_texto_ocr(texto):
    """
    Corrige texto OCR mantendo:
    - Indentação e bullets
    - Quebra de linhas
    - Hífens no final de linha
    - Siglas e termos técnicos
    """

```

```

    texto = normalizar_hifen_linhas(texto)

```

```

    matches = tool.check(texto)
    texto = language_tool_python.utils.correct(texto, matches)

```

```

    for errado, certo in CORRECOES_FIXAS.items():
        texto = re.sub(rf'\b{re.escape(errado)}\b', certo, texto)

```

```

    for errado, certo in CORRECOES_FRASES.items():
        texto = texto.replace(errado, certo)

```

```

    linhas = texto.splitlines()
    linhas_corrigidas = []

```

```

    for linha in linhas:
        palavras = re.findall(r'\b[\w\']+\b', linha)

```

```

        palavras_corrigidas = []

        for p in palavras:
            palavra_lower = p.lower()
            palavra_sem_acento = unidecode(palavra_lower)

            if len(p) <= 2 or p.isupper() or p.isdigit() or '-' in p:
                palavras_corrigidas.append(p)
                continue

            if p in TERMOS_ESPECIAIS or palavra_sem_acento in
TERMOS_ESPECIAIS SEM ACENTO:
                palavras_corrigidas.append(p)
                continue

            if palavra_lower in DICIONARIO:
                palavras_corrigidas.append(p)
                continue
            match_sem_acento = next((w for w in DICIONARIO if
unidecode(w) == palavra_sem_acento), None)
            if match_sem_acento:
                correcao = match_sem_acento
                if p[0].isupper():
                    correcao = correcao.capitalize()
                palavras_corrigidas.append(correcao)
                continue

            sugestoes = get_close_matches(palavra_sem_acento,
DICIONARIO, n=1, cutoff=0.9)
            if sugestoes:
                correcao = sugestoes[0]
                if p[0].isupper():
                    correcao = correcao.capitalize()
                palavras_corrigidas.append(correcao)
            else:
                palavras_corrigidas.append(p)

        espacos = re.match(r'^\s*', linha).group(0)
        linha_corrigida = espacos + ' '.join(palavras_corrigidas)
        linhas_corrigidas.append(linha_corrigida)

    return '\n'.join(linhas_corrigidas)

```

```

def ocr_pdf_para_texto(pdf_path):
    """
    Extrai texto de todas as páginas do PDF, aplica OCR quando
necessário
    e corrige erros ortográficos preservando parágrafos.
    """
    pdf = fitz.open(pdf_path)

```

```

    texto_completo = ""

    for i, pagina in enumerate(pdf):
        numero = i + 1
        print(f"Processando página {numero}...")

        # Extrai texto direto do PDF
        texto = pagina.get_text("text")

        # Se a página tiver pouco texto, aplica OCR
        if len(texto.strip()) < 600:
            pix = pagina.get_pixmap(dpi=250)
            img_path = os.path.join(TMPDIR, f"pagina_{numero}.png")
            pix.save(img_path)
            texto = extrair_texto_tesseract(img_path)
            os.remove(img_path)

        texto_completo += f"\n\n--- Página {numero} ---\n\n{texto}"

    pdf.close()

    # Corrige automaticamente erros de OCR e ortografia
    # texto_corrigido = corrigir_texto_ocr(texto_completo)
    texto_corrigido = corrigir_texto_ocr(texto_completo)
    texto_corrigido = corrigir_frases(texto_corrigido)

    return texto_corrigido.strip()

```

Explicação:

```
CACHE = {}
```

- Cria um dicionário vazio para armazenar resultados temporários de correções.

```

DICCIONARIO_URL = "https://www.ime.usp.br/~pf/dicios/br-utf8.txt"
print("♦ Baixando dicionário...")
r = requests.get(DICCIONARIO_URL)

```

```

DICCIONARIO_BASE = set(line.strip().lower() for line in
r.text.splitlines() if line.strip())

```

- Define a URL do dicionário oficial do IME-USP (contém milhares de palavras do português).
- Faz o download via requests.get().

- Limpa cada linha (removendo espaços e vazios), coloca tudo em minúsculas e converte em um set (conjunto) para busca rápida.
- Motivo técnico: o uso de set() torna a verificação de palavras O(1) (muito mais rápida).
- Isso é essencial para comparar milhares de palavras OCR em tempo real.

```
DICIONARIO_TECNICO = {
    "siggeo", "sig", "erpotech", "geotiff", "cad/gis", "qgis",
    "geoprocessamento", "geodésia", "geotecnologias",
    "georreferenciamento",
    "cartografia", "geoespacial", "teledetecção", "altimetria",
    "hidrografia", "topografia", "geocodificação",
    "georreferenciadas", "avanços", "divisão", "instalações",
    "unidade", "Seção", "empresas", "aculturamento"
}
```

- Conjunto de termos específicos da área de geoprocessamento e cartografia.

```
DICIONARIO = DICIONARIO_BASE.union(DICIONARIO_TECNICO)

print(f"[DICIONÁRIO] {len(DICIONARIO):,} palavras carregadas (incluindo técnicas).")
```

- Une o dicionário base e o técnico num só.

```
TERMOS_ESPECIAIS = {"SIGeo", "ERPTech", "GeoTIFF", "CAD/GIS", "QGIS"}
TERMOS_ESPECIAIS_SEM_ACENTO = set(unidecode(t.lower()) for t in
TERMOS_ESPECIAIS)

TERMOS_APRENDIDOS = set()
```

- Define siglas e nomes próprios que devem sempre ser preservados.
- Gera uma versão sem acento (unidecode) para comparação robusta.
- TERMOS_APRENDIDOS guarda palavras que o sistema corrige dinamicamente com o tempo.

As estruturas CORRECOES_FIXAS e CORRECOES_FRASES são mapas de substituição direta, criadas a partir da observação de erros recorrentes do Tesseract.

```
def normalizar_hifen_linhas(texto):
    """
    Junta palavras quebradas por hífen no final da linha, preservando
    indentação e bullets.
```

```

"""
linhas = texto.splitlines()
linhas_corrigidas = []
i = 0
while i < len(linhas):
    linha = linhas[i]
    if linha.rstrip().endswith('-') and i + 1 < len(linhas):
        prox = linhas[i + 1].lstrip()
        linha = linha.rstrip()[:-1] + prox
        i += 1
    linhas_corrigidas.append(linha)
    i += 1

return '\n'.join(linhas_corrigidas)

```

- juntar palavras que foram separadas por hífen no final de uma linha (hifenização de quebra de linha)

```

CORRECOES_FIXAS_EXPANDIDO = {}
for k, v in CORRECOES_FIXAS.items():
    CORRECOES_FIXAS_EXPANDIDO[k] = v
    sem_acento = unicode(k)
    if sem_acento != k:
        CORRECOES_FIXAS_EXPANDIDO[sem_acento] = v

```

```

CORRECOES_FIXAS = CORRECOES_FIXAS_EXPANDIDO

```

CORRECOES_FIXAS_EXPANDIDO:

- cria novo dicionário vazio.

```

for k, v in CORRECOES_FIXAS.items():

```

- itera pares (errado → certo).

```

CORRECOES_FIXAS_EXPANDIDO[k] = v

```

- adiciona a correção original.

```

sem_acento = unicode(k)

```

- obtém versão sem acento de k (ex.: "Seção" → "Secao").

```

if sem_acento != k:

```

- só adiciona a versão sem acento se ela for diferente (evita redundância).

```
CORRECOES_FIXAS_EXPANDIDO[sem_acento] = v
```

- adiciona a chave sem acento apontando para mesma correção.

```
CORRECOES_FIXAS = CORRECOES_FIXAS_EXPANDIDO
```

- troca o dicionário original pela versão expandida.

```
def distancia levenshtein(a, b):  
    if a == b: return 0  
    if len(a) == 0: return len(b)  
    if len(b) == 0: return len(a)  
    matrix = [[0]*(len(b)+1) for _ in range(len(a)+1)]  
    for i in range(len(a)+1): matrix[i][0] = i  
    for j in range(len(b)+1): matrix[0][j] = j  
    for i in range(1, len(a)+1):  
        for j in range(1, len(b)+1):  
            custo = 0 if a[i-1] == b[j-1] else 1  
            matrix[i][j] = min(matrix[i-1][j]+1, matrix[i][j-1]+1,  
matrix[i-1][j-1]+custo)  
  
    return matrix[len(a)][len(b)]
```

Tratos iniciais (checar igualdade ou strings vazias) para retornar resultados triviais e evitar computação desnecessária.

```
matrix = [[0]*(len(b)+1) for _ in range(len(a)+1)]
```

- cria tabela $(m+1) \times (n+1)$.

Inicializa primeira coluna/linha com índices (custo de transformar string vazia em prefixos).

Loop duplo preenche a matriz usando relação de recorrência:

```
custo = 0 if a[i-1] == b[j-1] else 1
```

- custo de substituição.

```
matrix[i][j] = min(...)
```

- escolha entre inserção, deleção ou substituição.

Uso prático: ajuda a decidir se uma palavra OCRizada é próxima de uma palavra do dicionário. Em seu código você usa `get_close_matches` (difflib) que já oferece

heurística, mas ter Levenshtein implementado permite criar regras customizadas (ex.: aceitar correções só se distância ≤ 2).

```
def aplicar_acentos(original, corrigida):
    resultado = ""
    for o, c in zip(original, corrigida):
        if re.match(r'[áéíóúãõâêôâèìòùç]', o.lower()):
            resultado += o
        else:
            resultado += c
    resultado += corrigida[len(resultado):]

    return resultado
```

- preserva acentuação caso esteja presente na palavra original extraída pelo OCR quando apropriado — útil se a palavra corrigida perdeu acento que o OCR havia recuperado corretamente em alguns caracteres.

```
def corrigir_frases(texto):
    for errado, certo in CORRECOES_FRASES.items():
        if errado in texto:
            texto = texto.replace(errado, certo)
            print(f"♦ Frase corrigida: '{errado}' -> '{certo}'")

    return texto
```

- aplica substituições completas de trechos/frases que o OCR costuma gerar erradamente, como casos fixos onde a correção depende de contexto maior que palavra por palavra.

```
def corrigir_texto_ocr(texto):
    """
    Corrige texto OCR mantendo:
    - Indentação e bullets
    - Quebra de linhas
    - Hífens no final de linha
    - Siglas e termos técnicos
    """

    texto = normalizar_hifen_linhas(texto)

    matches = tool.check(texto)
    texto = language_tool_python.utils.correct(texto, matches)

    for errado, certo in CORRECOES_FIXAS.items():
        texto = re.sub(rf'\b{re.escape(errado)}\b', certo, texto)
```

```

    for errado, certo in CORRECOES_FRASES.items():
        texto = texto.replace(errado, certo)

    linhas = texto.splitlines()
    linhas_corrigidas = []

    for linha in linhas:
        palavras = re.findall(r'\b[\w\']+\b', linha)
        palavras_corrigidas = []

        for p in palavras:
            palavra_lower = p.lower()
            palavra_sem_acento = unidecode(palavra_lower)

            if len(p) <= 2 or p.isupper() or p.isdigit() or '-' in p:
                palavras_corrigidas.append(p)
                continue

            if p in TERMOS_ESPECIAIS or palavra_sem_acento in
TERMOS_ESPECIAIS SEM ACENTO:
                palavras_corrigidas.append(p)
                continue

            if palavra_lower in DICIONARIO:
                palavras_corrigidas.append(p)
                continue

            match_sem_acento = next((w for w in DICIONARIO if
unidecode(w) == palavra_sem_acento), None)
            if match_sem_acento:
                correcao = match_sem_acento
                if p[0].isupper():
                    correcao = correcao.capitalize()
                palavras_corrigidas.append(correcao)
                continue

            sugestoes = get_close_matches(palavra_sem_acento,
DICIONARIO, n=1, cutoff=0.9)
            if sugestoes:
                correcao = sugestoes[0]
                if p[0].isupper():
                    correcao = correcao.capitalize()
                palavras_corrigidas.append(correcao)
            else:
                palavras_corrigidas.append(p)

        espacos = re.match(r'^\s*', linha).group(0)
        linha_corrigida = espacos + ' '.join(palavras_corrigidas)
        linhas_corrigidas.append(linha_corrigida)

    return '\n'.join(linhas_corrigidas)

```

```
texto = normalizar_hifen_linhas(texto)
```

- Hífens quebram palavras que mudariam a tokenização. Se juntarmos depois da correção gramatical, a LanguageTool ou substituições poderiam tratar a palavra como duas separadas, perdendo contexto. Portanto normalizar hífen antes garante tokens corretos.

```
matches = tool.check(texto)
```

```
texto = language_tool_python.utils.correct(texto, matches)
```

Executa LanguageTool (correção gramatical/ortográfica contextual). Faz isso antes das correções fixas e do dicionário por dois motivos:

- LanguageTool consegue reparar problemas sintáticos complexos (concordância, preposição, dupla palavra), melhorando a estrutura geral.
- Aplicar LanguageTool antes evita que substituições fixas se conflitem com sugestões gramaticais que dependem de contexto de frase.

```
for errado, certo in CORRECOES_FIXAS.items():
```

```
    texto = re.sub(rf'\b{re.escape(errado)}\b', certo, texto)
```

- Aplica substituições diretas por palavra (usa \b para marcar limites de palavra e re.escape para evitar que caracteres especiais causem regex inválida).
- Essas correções pegam casos persistentes que nem sempre LanguageTool corrige (erros típicos do OCR).

```
for errado, certo in CORRECOES_FRASES.items():
```

```
    texto = texto.replace(errado, certo)
```

- Aplica correções de frase inteira (podem ter caracteres que não se encaixam em \b), consertando trechos inteiros.

```
linhas = texto.splitlines()
```

```
linhas_corrigidas = []
```

- Agora que a estrutura global está mais estável, vamos tratar palavra a palavra preservando linhas e formatação.

```
for linha in linhas:
    palavras = re.findall(r'\b[\w\']+\b', linha)

    palavras_corrigidas = []
```

- `re.findall(r'\b[\w\']+\b', linha)` extrai tokens (palavras, permitindo apóstrofes e hífen internos). Essa regex ignora pontuações externas (vírgulas, pontos).
- Construímos lista de palavras corrigidas por linha.

```
for p in palavras:
    palavra_lower = p.lower()
    palavra_sem_acento = unidecode(palavra_lower)

    if len(p) <= 2 or p.isupper() or p.isdigit() or '-' in p:
        palavras_corrigidas.append(p)

    continue
```

Pré-filtros — preserva:

- palavras muito curtas (`<=2`): artigos/preposições, risco de “correção equivocada”;
- palavras totalmente em maiúsculas (siglas → não as altera);
- tokens numéricos (IDs, datas);
- palavras com - (hifenadas ou códigos) — evita corrigi-las automaticamente;
- Esses filtros reduzem falsos positivos.

```
if p in TERMOS_ESPECIAIS or palavra_sem_acento in
TERMOS_ESPECIAIS_SEM_ACENTO:
    palavras_corrigidas.append(p)

    continue
```

- Preserva termos especiais (lista de siglas/nomes técnicos), comparando também versão sem acento (por segurança).

```
if palavra_lower in DICIONARIO:
    palavras_corrigidas.append(p)
```

```
continue
```

- Se a palavra já existe no dicionário (minúscula), mantemos como está — confiança alta.

```
match_sem_acento = next((w for w in DICCIONARIO if unicode(w) ==  
palavra_sem_acento), None)  
if match_sem_acento:  
    correcao = match_sem_acento  
    if p[0].isupper():  
        correcao = correcao.capitalize()  
    palavras_corrigidas.append(correcao)
```

```
continue
```

- Tenta encontrar uma palavra no dicionário cuja forma sem acento seja igual à token OCRizada sem acento.
- Isso recupera acentos perdidos de forma confiável.

```
sugestoes = get_close_matches(palavra_sem_acento, DICCIONARIO, n=1,  
cutoff=0.9)  
if sugestoes:  
    correcao = sugestoes[0]  
    if p[0].isupper():  
        correcao = correcao.capitalize()  
    palavras_corrigidas.append(correcao)  
else:
```

```
    palavras_corrigidas.append(p)
```

- Quando não há correspondência sem acento, usa `diffib.get_close_matches` para obter a palavra mais próxima (cutoff 0.9 = alta similaridade exigida).
- Se encontrar sugestão, aplica (ajustando capitalização), caso contrário mantém token original (preferível a uma "correção errada").

```
espacos = re.match(r'^\s*', linha).group(0)  
linha_corrigida = espacos + ' '.join(palavras_corrigidas)
```

```
linhas_corrigidas.append(linha_corrigida)
```

- Recupera os espaços iniciais da linha (`^\s*`) para preservar indentação (importante para bullets, níveis de tópicos).

- Rejunta as palavras com um único espaço (normaliza múltiplos espaços internos).
- Acumula a linha corrigida.

```
return '\n'.join(linhas_corrigidas)
```

- Reconstrói o texto final com quebras de linha preservadas.

```
def ocr_pdf_para_texto(pdf_path):
    """
    Extrai texto de todas as páginas do PDF, aplica OCR quando
    necessário
    e corrige erros ortográficos preservando parágrafos.
    """
    pdf = fitz.open(pdf_path)
    texto_completo = ""

    for i, pagina in enumerate(pdf):
        numero = i + 1
        print(f"Processando página {numero}...")

        # Extrai texto direto do PDF
        texto = pagina.get_text("text")

        # Se a página tiver pouco texto, aplica OCR
        if len(texto.strip()) < 600:
            pix = pagina.get_pixmap(dpi=250)
            img_path = os.path.join(TMPDIR, f"pagina {numero}.png")
            pix.save(img_path)
            texto = extrair_texto_tesseract(img_path)
            os.remove(img_path)

        texto_completo += f"\n\n--- Página {numero} ---\n\n{texto}"

    pdf.close()

    # Corrige automaticamente erros de OCR e ortografia
    # texto corrigido = corrigir_texto_ocr(texto_completo)
    texto_corrigido = corrigir_texto_ocr(texto_completo)
    texto_corrigido = corrigir_frases(texto_corrigido)

    return texto_corrigido.strip()

pdf = fitz.open(pdf_path)
```

- Abre o PDF usando PyMuPDF (fitz).

- pdf passa a ser um objeto representando o documento; permite iterar páginas, extrair texto nativo ou renderizar a página como imagem.

```
texto_completo = ""
```

- Inicializa a string que irá acumular o conteúdo processado de todas as páginas. Usar uma string é simples e direto; para documentos muito grandes poderia usar list + ".join()" ao final por eficiência, embora aqui você já concatene por páginas (a diferença é pequena).

```
for i, pagina in enumerate(pdf):  
    numero = i + 1
```

```
    print(f"Processando página {numero}...")
```

- Loop que percorre cada página do PDF.
- enumerate(pdf) fornece índice i (0-based) e o objeto pagina.
- numero = i + 1 deixa a numeração humana (1-based).
- print(...) é log/feedback simples para acompanhar progresso no terminal — útil ao processar lotes longos.

```
texto = pagina.get_text("text")
```

- Tenta extrair texto nativo da página (camada de texto do PDF).
- Quando o PDF foi gerado digitalmente (não escaneado), get_text("text") geralmente retorna texto perfeito, com ordenação correta e sem necessidade de OCR.
- Isso é muito mais rápido e preciso que aplicar OCR em tudo.

```
if len(texto.strip()) < 600:
```

Heurística: se o texto extraído for curto (menos de 600 caracteres), assume-se que a página provavelmente é uma imagem escaneada ou tem pouco texto nativo (por exemplo uma figura, uma tabela como imagem).

texto.strip():

- remove espaços em branco; o comprimento real indica presença de conteúdo.

```
pix = pagina.get_pixmap(dpi=250)
```

- Renderiza a página como imagem (Pixmap) com 250 DPI.
- 250 DPI é um bom trade-off: aumenta legibilidade para OCR sem explodir o tamanho do arquivo; valores típicos para OCR ficam entre 200–300 DPI.

```
img_path = os.path.join(TMPDIR, f"pagina {numero}.png")
```

- Monta o caminho do arquivo temporário onde a imagem será salva.
- TMPDIR deve ser uma pasta temporária existente (definida globalmente).
- Usar os.path.join garante compatibilidade de caminho entre sistemas operacionais.

```
pix.save(img_path)
```

- Salva a imagem (PNG) no disco. O Tesseract normalmente consome arquivos ou objetos PIL, então salvar temporariamente é prático.

```
texto = extrair_texto_tesseract(img_path)
```

- Chama a função extrair_texto_tesseract, que pré-processa a imagem (filtros, CLAHE, binarização) e executa pytesseract.image_to_string.
- Resultado é a string OCR para a página renderizada.

```
os.remove(img_path)
```

- Remove o arquivo temporário para não acumular lixo em disco. Boa prática de limpeza.

```
texto_completo += f"\n\n--- Página {numero} ---\n\n{texto}"
```

- Concatena ao texto_completo um separador visual por página e o texto da página.
- A marcação --- Página X --- facilita debugging, revisões manuais e mapeamento do texto de volta à página original.

```
pdf.close()
```

- Fecha o documento PDF liberando recursos. Boa prática fechar handle de arquivo.

```
texto_corrigido = corrigir_texto_ocr(texto_completo)
```

```
texto_corrigido = corrigir_frases(texto_corrigido)
```

Pós-processamento em duas etapas:

- `corrigir_texto_ocr(texto_completo)` — aplica normalização de hífen, LanguageTool, correções fixas, dicionário e matching palavra-a-palavra (o motor de correção que discutimos).
- `corrigir_frases(texto_corrigido)` — aplica substituições de frases inteiras (CORRECOES_FRASES) e imprime logs.

```
return texto_corrigido.strip()
```

- Retorna o texto final, com `strip()` removendo espaços/linhas extras nas extremidades.
- Resultado pronto para armazenamento, indexação ou exportação.

Dessa forma o resultado vem de forma correta:

```
PS C:\xampp\htdocs\leis_externo> python
C:\xampp\htdocs\leis_externo\processamento_OCR.py
>>
♦ Baixando dicionário...
[DICIONÁRIO] 261,495 palavras carregadas (incluindo técnicas).

♦ Processando lote a partir do ID 21312...
Processando página 1...
Processando página 2...
Processando página 3...
Processando página 4...
♦ Frase corrigida: 'Profissão eximias' -> 'Profissão exigida'
♦ Frase corrigida: 'suplementavas se necessário' -> 'suplementadas se necessário'
✔ Lei 21312 atualizada
🏁 Lote ID 21312 finalizado!
```

```
🏆 Todos os lotes processados!
```

Resultado:

Página 1

Fonte Departamento de Relações Administrativas Prefeitura de Guarulhos

DECRETO n ° 19542

de 27 de agosto de 1996

Dispõe Sobre Define atribuições de sub-unidades
criadas pelo Decreto n ° 19 533 96

O BACHAREL VICENTINO PAPOTITO PREFEITO DE
GUARULHOS no uso de suas atribuições legais
Considerando que compete a Secretaria de Economia e Planejamento
a definição de metodologia que garantam o planejamento e gerenciamento
do Município
para adequar suas aptidões sempre em sintonia com os mais modernos
recursos
tecnológicos
Considerando a necessidade de modernização das rotinas de
procedimentos técnicos-administrativos voltados a otimização quanto a
prestação de
serviços
Considerando os investimentos já implementados e dirigidos as
primeiras fases de implantação do projeto SIGeo
Considerando a realidade quanto a utilização desta tecnologia junto as
várias unidades administrativas desta Prefeitura já incorporados as
suas rotinas de
trabalho
Considerando a existência de produtos já gerados e implantados pela
Comissão Técnica Multidisciplinar formada por técnicos desta Secretaria
Considerando o aprimoramento técnico desta equipe detendo
conhecimentos específicos nas áreas envolvidas
Considerando a existência de um parque tecnológico de equipamentos
de última geração voltados as áreas de informática e geotecnologias
geoprocessamento
geodésia cartografia topografia entre outros
Considerando a necessidade de manter continuamente atualizada a
base cartográfica do município assim como também tornar compatível e
confiável seu
banco de dados gráfico digital e tabular entre outros
Considerando a necessidade de garantir a institucionalização da
estrutura do projeto SIGeo visando assegurar a continuidade deste de
forma que sejam
implantadas e desenvolvidas todas as etapas previstas e
Considerando o artigo 29 da Lei n ° 8 214 91 e o que consta do
processo n ° 17 092 96

DECRETA

Artigo 1º Ficam definidas as atribuições específicas das subunidades da Secretaria de Economia e Planejamento criadas através do Decreto nº 19.533/96 conforme segue

Página 2

Denominação da
unidade Divisão Técnica do SIGeo
Sistema de Informações Georreferenciadas

Condição de provimento
Nível Universitário com especializa
ção em Geoprocessamento ou reconhecida
experiência na área

Profissão exigida
Engenheiro Geólogo Geógrafo ou Arquiteto

Atribuições específicas
Gerenciar o Sistema Cartográfico Municipal SCM GUL
Normatizar e padronizar a execução de levantamentos
topográficos cartográficos e geodésicos do Município

Gerir a implantação e desenvolvimento do Sistema de Informação
Georreferenciadas de Guarulhos SIGeo-Guarulhos

Promover o aprimoramento técnico-científico da equipe
multidisciplinar responsável pela condução do Sigeo-Guarulhos
visando o desenvolvimento de todas as etapas do Projeto de forma
adequada e atrelada aos avanços tecnológicos disponíveis no
mercado x

Estabelecer as diretrizes para a determinação das instalações e
equipamentos adequados a todas as etapas do Projeto

Definir os sistemas aplicados ao geoprocessamento

Divulgar o SIGeo junto as empresas privadas ou publicas
prestaras de serviços prevendo o intercâmbio de informações de
interesse a administração municipal

Elaborar as especificações técnicas dos Termos de Referência
Visando a contratação de serviços na área de topografia
cartografia geodésia e geoprocessamento em projetos correlatos
Bem como acompanhar os serviços e ou projetos contratados
visando o seu controle de qualidade e
Arbitrar e propor parecer sobre questões fronteiras Quanto
aos limites do território do Município de Guarulhos

Denominação da
unidade Seção Técnica de Geoprocessamento

Condição de provimento Profissão exigida

Nível Universitário com especializa Engenheiro Geólogo Geocao em
Geoprocessamento ou reconhecida geógrafo ou Arquiteto
experiência na área

Atribuições específicas

Levantar o fluxo de informações entre as unidades administrai
administrativas
Promover a concepção e desenvolvimento de sistemas compatíveis
com as necessidades técnico-administrativas
Disseminar o uso da tecnologia de Geoprocessamento junto as
unidades administrativas Prevendo o aculturamento treinamento

e acompanhamento 1

Fonte Departamento de Relações Administrativas Prefeitura de Guarulhos

Página 3

Estabelecer o estruturar as rotinas de manutenção das bases doe
dados gráficos e alfanuméricos

Gerenciar a ligação e compatibilizava das bases de dados gráficos
gráficos e alfanuméricos

Promover a geração de novas informações pelo cruzamento de da-r
dos existentes abrangendo avaliações espaciais e temporada
a fim de alimentar o Sistemas

Gerir e manter a operacionalidade do Sistema e

Promover a aquisição de informações alfanuméricas sociais demográficas
econômicas e etc das mais variadas fontes junto as
instituições provedorias de tais dados para o posterior tratamento
to e compatibilizava a fim de carregá-las no Sistema
Denominação da

unidade Seção Técnica de Geociências

Condição de provimento Profissão exigida

Nível Universitário com especializa Engenheiro Geólogo Geógrafo
cão em Geoprocessamento ou reconhecida grafo ou Arquiteto
experiência na área

Atribuições específicas

Manter a base cartográfica digital do Município continuamente atualizadas

Zelar pela guarda Controle e manutenção de todo acervo cartográfico gerado a partir do mapeamento digital do município bem como o acervo aerofotográfico a ele relacionados

Promover a manutenção da rede básica de apoio geodésico

Adensar a rede básica de apoio geodésico a fim de prover novas áreas ou melhorar as condições de realização dos trabalhos topográficos

Fornecer e dar atendimento as solicitações de referências de amarração para a realização de levantamentos topográficos Na forma de monografias de marcos da rede básica de apoio geodésico z Analisar os produtos gerados em trabalhos das áreas de topografia Cartografia e geodésia realizados no Município para posterior migração para a base cartográfica oficial x

Promover a manutenção sistemática e guarda dos equipamentos empregados na manutenção e densificação da rede básica de apoio geodésicos

Produzir através de edição recorte e função de plantas da base cartográfica do Município plantas em outras escalas e ou formatos específicos E

Elaborar através da conversão de mapas temáticos existentes Mapas no formato digital para carga e alimentação do Sistema

Artigo 2º As despesas decorrentes do presente Decreto correrão por conta de dotações próprias do orçamento vigente suplementadas se necessário

Artigo 3º Este decreto entra em vigor na data de sua publicação revogadas as disposições em contrário

Fonte Departamento de Relações Administrativas Prefeitura de Guarulhos

Página 4

Fonte Departamento de Relações Administrativas Prefeitura de Guarulhos

Guarulhos 27 de agosto de 1996

BEL VICENTINO PAPOTITO
Prefeito

Dr NEWTON DE OLIVEIRA VANS
Secretário de Administração

Registrado no Departamento de Relações Administrativas Secretaria do
Governo
Municipal da Prefeitura Municipal de Guarulhos e afixado no lugar
público de costume aos
vinte e sete dias do mês de agosto de mil novecentos e noventa e seis

DEISE RAMPADO
Diretora

Publicado no Jornal Folha Metropolitana em 28 de agosto de 1996

REVOGADO PELO DECRETO n ° 22572 2004

Este artigo apresenta o desenvolvimento e aperfeiçoamento de um sistema automatizado para extração de texto a partir de documentos digitalizados, empregando técnicas de Optical Character Recognition (OCR). O objetivo principal é aumentar a confiabilidade da leitura de textos provenientes de arquivos PDF com baixa qualidade de digitalização, presença de marcas d'água e distorções gráficas. A solução foi desenvolvida em Python, utilizando as bibliotecas Tesseract OCR, OpenCV e PyMuPDF (Poppler), com módulos complementares de correção ortográfica e semântica automática. São discutidos os desafios encontrados na digitalização de documentos administrativos e os ganhos obtidos com a aplicação de pré-processamento de imagem e otimização de parâmetros do Tesseract.

Introdução

O reconhecimento óptico de caracteres (OCR) é uma tecnologia essencial para converter imagens ou PDFs digitalizados em texto pesquisável. No entanto, a

qualidade do resultado depende diretamente da nitidez da imagem, presença de ruídos, marca d'água e tipo de fonte usada no documento original.

Neste artigo, mostraremos como otimizar o processo de OCR com Python usando Tesseract OCR, Poppler e OpenCV, aplicando técnicas de pré-processamento de imagem que melhoram significativamente a precisão do texto reconhecido — principalmente em documentos com baixa qualidade de digitalização ou marcas d'água.

Problema Real:

Durante os testes iniciais com o Tesseract OCR (antes da automação em Python), alguns registros — como o de ID 21312 — apresentavam trechos ilegíveis e palavras desconfiguradas devido à baixa resolução, sombras e marca d'água sobre o texto.

Exemplo de resultado anterior sem pré-processamento:

DECRETO N° 19542

de 27 de agosto de 1996

Dispõe Sobre: "Define atri s de sub-unidades

MUNICIPAL DE

ê Economia e Planejamento

ento e gerenciamento do Município

E em sintonia com os mais modernos recursos

Percebe-se que o OCR não conseguiu interpretar corretamente as palavras ("atri s", "ento", "E em") nem manter a formatação dos parágrafos.

Esses erros eram recorrentes em documentos com:

1. Baixo contraste entre texto e fundo;
2. Marcas d'água transparentes;
3. Imagens digitalizadas tortas ou com manchas.

Método:

Para solucionar esses problemas, foi desenvolvido um pipeline completo em Python com três etapas principais:

1. Pré-processamento das imagens (para aumentar contraste e remover ruídos);
2. Extração de texto com o Tesseract OCR;
3. Correção automática de erros ortográficos e reconstrução de parágrafos.

```
def preprocess_image(img_path):  
    """Pré-processa imagem para OCR mais confiável"""  
    img = cv2.imread(img_path)  
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  
    gray = cv2.bilateralFilter(gray, 9, 75, 75)  
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))  
    gray = clahe.apply(gray)  
    thresh = cv2.adaptiveThreshold(  
        gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
        cv2.THRESH_BINARY, 15, 8  
    )  
  
    processed_path = img_path.replace(".png", "_proc.png")  
    cv2.imwrite(processed_path, thresh)  
  
    return processed_path
```

Explicação:

```
img = cv2.imread(img_path)
```

- Lê a imagem original do PDF convertida em PNG.

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

- Converte para tons de cinza, reduzindo o impacto das cores do fundo e marca d'água.

```
gray = cv2.bilateralFilter(gray, 9, 75, 75)
```

- Aplica filtro bilateral, que remove ruídos sem borrar as bordas das letras — essencial para OCR.

```
clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
```

- Cria o algoritmo CLAHE (Equalização de Histograma Adaptativa), que aumenta o contraste localmente.

```
gray = clahe.apply(gray)
```

- Aplica o CLAHE, realçando letras apagadas ou sobre fundo manchado.

```
thresh = cv2.adaptiveThreshold(...)
```

- Cria uma imagem binária (preto e branco) adaptando o limiar de contraste por região, o que elimina sombras e marcas d'água leves.

```
cv2.imwrite(processed_path, thresh)
```

- Salva a imagem pré-processada que será usada pelo OCR.

```
def extrair_texto_tesseract(img_path):
    """Extrai texto da imagem usando Tesseract OCR"""
    img_proc = preprocess_image(img_path)
    pil_img = Image.open(img_proc)
    texto_final = pytesseract.image_to_string(pil_img, lang='por',
    config='--oem 3 --psm 4')
    os.remove(img_proc)

    return texto_final
```

Explicação:

```
img_proc = preprocess_image(img_path)
```

- Passa a imagem pelo filtro e equalização antes do OCR.

```
pytesseract.image_to_string(..., lang='por')
```

- Extrai o texto em português, respeitando acentuação e regras linguísticas.

```
config='--oem 3 --psm 4'
```

- Define o modo OEM 3 (OCR Engine Mode: LSTM + Legacy) e PSM 4 (análise de texto em blocos), ideal para documentos com múltiplas colunas.

```
os.remove(img_proc)
```

- Exclui a imagem temporária processada após a leitura.

Mesmo com tudo isso implementado ainda sim vinha muito resultado com erros de ortografia nas palavras e também de forma semântica vinha de forma errada, como:

```
Divisadã Técnica de Georáficas
Profissão eximias: Georáfo
Sagrou-Guarulhos (SIGNO)
```

```
Geoprocessamento- dos dados
```

Por isso foi pensado em utilizar se dos recursos disponíveis para melhorar ainda mais a qualidade, no caso:

Método:

```
CACHE = {}

DICIONARIO_URL = "https://www.ime.usp.br/~pf/dicios/br-utf8.txt"
print("♦ Baixando dicionário...")
r = requests.get(DICIONARIO_URL)
DICIONARIO_BASE = set(line.strip().lower() for line in
r.text.splitlines() if line.strip())

DICIONARIO_TECNICO = {
    "siggeo", "sig", "erptech", "geotiff", "cad/gis", "qgis",
    "geoprocessamento", "geodésia", "geotecnologias",
    "georreferenciamento",
    "cartografia", "geoespacial", "teledetecção", "altimetria",
    "hidrografia", "topografia", "geocodificação",
    "georreferenciadas", "avanços", "divisão", "instalações",
    "unidade", "Seção", "empresas", "aculturamento"
}

DICIONARIO = DICIONARIO_BASE.union(DICIONARIO_TECNICO)
print(f"[DICIONÁRIO] {len(DICIONARIO):,} palavras carregadas (incluindo
técnicas).")

TERMOS_ESPECIAIS = {"SIGeo", "ERPTech", "GeoTIFF", "CAD/GIS", "QGIS"}
TERMOS_ESPECIAIS_SEM_ACENTO = set(unidecode(t.lower()) for t in
TERMOS_ESPECIAIS)
TERMOS_APRENDIDOS = set()

CORRECOES_FIXAS = {
    "Divisadão": "Divisão",
    "Sintema": "Sistema",
```

```
"Municípios": "Município",
"à": "a",
"Sagrou-Guarulhos": "Sigeo-Guarulhos",
"anos": "aos",
"asas": "as",
"Projetos": "Projeto",
"socâmo": "Seção",
"Socão": "Seção",
"arca": "área",
"laorafo": "geógrafo",
"vVAass": "administrativas",
"Socâmo": "Seção",
"AA": "a",
"Carlos-": "cartográfico",
"toronja-": "topografia",
"Torna-r": "formatos",
"alunização": "alimentação",
"alunização": "alimentação",
"suplementavas": "suplementadas",
"BEL.": "Bacharel",
"cão": "ção",
"especificas": "específicas",
"experiência na arca": "experiência na área",
"Geo-": "Geo",
"grafico": "gráfico",
"grã-r": "gráficos",
"da'": "da",
"profissão eximias": "profissão exigida",
"650": "Geógrafo",
"Gerir à": "Gerir a",
"às": "as",
"Signo": "SIGeo",
"SRH/SUL": "SCM/GUL", # manter
"projétos": "projetos",
"Geociências": "Geociências", # manter
"informação": "Informação",
"Georreferenciadas": "Georreferenciadas",
"Referência".lower(): "referência",
"Divisadã Técnica": "Divisão Técnica",
"Socâmo Técnica": "Seção Técnica",
"Geoprocessamento-": "Geoprocessamento",
"Carlos-gráfico": "cartográfico",
"marcos": "marcos",
"posterior migração": "posterior migração",
"suplementavas": "suplementadas",
"Geográficas": "Geográficas",
"de-": "de ",
"Grã-r": "Gráficos",
"Quadra-da": "Qualidade",
"qualidade, e |": "qualidade, e",
"topo -": "topográficos",
```

```

    "geo-r": "geodésicos",
    "Geotecnologias": "Geotecnologias"
}

CORRECOES FRASES = {
    "Denominação da' |": "Denominação da",
    "Nível Universitário.com especializa-": "Nível Universitário com especialização",
    "Engenheiro, Geólogo. Geo- grafo ou Arquiteto": "Engenheiro, Geólogo, Geógrafo ou Arquiteto",
    "Georreferenciadas de Guarulhos (SIGNO-Guarulhos)": "Georreferenciadas de Guarulhos (SIGeo-Guarulhos)",
    "adequada e atrelada.aos": "adequada e atrelada aos",
    "geoprocessamento;s": "geoprocessamento;",
    "empresas privadas ou publicas prestaras de serviços": "empresas privadas ou públicas prestadoras de serviços",
    "questões fronteiríços": "questões fronteiriças",
    "administraí-administrativas": "administrativas",
    "Bases doe dados gráficos": "bases de dados gráficos",
    "ficou e alfanuméricos": "gráficos e alfanuméricos",
    "cruzamento de da-r dos existentes": "cruzamento dos dados existentes",
    "a fim doe carregá-las": "a fim de carregá-las",
    "Profissão eximias": "Profissão exigida",
    "continuamente atualizadas": "continuamente atualizada",
    "Carlos- gráfico": "cartográfico",
    "rede básica de apoio geodésico;s": "rede básica de apoio geodésico;",
    "toronja- fia": "topografia",
    "posterior migração": "posterior migração",
    "formatos tos' específicos": "formatos específicos",
    "suplementavas se necessário": "suplementadas se necessário"
}

```

```

def normalizar_hifen_linhas(texto):
    """
    Junta palavras quebradas por hífen no final da linha, preservando
    indentação e bullets.
    """
    linhas = texto.splitlines()
    linhas_corrigidas = []
    i = 0
    while i < len(linhas):
        linha = linhas[i]
        if linha.rstrip().endswith('-') and i + 1 < len(linhas):
            prox = linhas[i + 1].lstrip()
            linha = linha.rstrip()[:-1] + prox
            i += 1
        linhas_corrigidas.append(linha)

```

```

        i += 1
    return '\n'.join(linhas_corrigidas)

CORRECOES_FIXAS_EXPANDIDO = {}
for k, v in CORRECOES_FIXAS.items():
    CORRECOES_FIXAS_EXPANDIDO[k] = v
    sem_acento = unidecode(k)
    if sem_acento != k:
        CORRECOES_FIXAS_EXPANDIDO[sem_acento] = v

CORRECOES_FIXAS = CORRECOES_FIXAS_EXPANDIDO

def distancia_levenshtein(a, b):
    if a == b: return 0
    if len(a) == 0: return len(b)
    if len(b) == 0: return len(a)
    matrix = [[0]*(len(b)+1) for _ in range(len(a)+1)]
    for i in range(len(a)+1): matrix[i][0] = i
    for j in range(len(b)+1): matrix[0][j] = j
    for i in range(1, len(a)+1):
        for j in range(1, len(b)+1):
            custo = 0 if a[i-1] == b[j-1] else 1
            matrix[i][j] = min(matrix[i-1][j]+1, matrix[i][j-1]+1,
matrix[i-1][j-1]+custo)
    return matrix[len(a)][len(b)]

def aplicar_acentos(original, corrigida):
    resultado = ""
    for o, c in zip(original, corrigida):
        if re.match(r'[áéíóúãõâêôàèìòùç]', o.lower()):
            resultado += o
        else:
            resultado += c
    resultado += corrigida[len(resultado):]
    return resultado

def corrigir_frases(texto):
    for errado, certo in CORRECOES_FRASES.items():
        if errado in texto:
            texto = texto.replace(errado, certo)
            print(f"♦ Frase corrigida: '{errado}' -> '{certo}'")
    return texto

def corrigir_texto_ocr(texto):
    """
    Corrige texto OCR mantendo:
    - Indentação e bullets
    - Quebra de linhas
    - Hífens no final de linha
    - Siglas e termos técnicos
    """

```

```

    texto = normalizar_hifen_linhas(texto)

    matches = tool.check(texto)
    texto = language_tool_python.utils.correct(texto, matches)

    for errado, certo in CORRECOES_FIXAS.items():
        texto = re.sub(rf'\b{re.escape(errado)}\b', certo, texto)

    for errado, certo in CORRECOES_FRASES.items():
        texto = texto.replace(errado, certo)

    linhas = texto.splitlines()
    linhas_corrigidas = []

    for linha in linhas:
        palavras = re.findall(r'\b[\w\']+\b', linha)
        palavras_corrigidas = []

        for p in palavras:
            palavra_lower = p.lower()
            palavra_sem_acento = unidecode(palavra_lower)

            if len(p) <= 2 or p.isupper() or p.isdigit() or '-' in p:
                palavras_corrigidas.append(p)
                continue

            if p in TERMOS_ESPECIAIS or palavra_sem_acento in
TERMOS_ESPECIAIS_SEM_ACENTO:
                palavras_corrigidas.append(p)
                continue

            if palavra_lower in DICIONARIO:
                palavras_corrigidas.append(p)
                continue

            match_sem_acento = next((w for w in DICIONARIO if
unidecode(w) == palavra_sem_acento), None)
            if match_sem_acento:
                correcao = match_sem_acento
                if p[0].isupper():
                    correcao = correcao.capitalize()
                palavras_corrigidas.append(correcao)
                continue

            sugestoes = get_close_matches(palavra_sem_acento,
DICIONARIO, n=1, cutoff=0.9)
            if sugestoes:
                correcao = sugestoes[0]
                if p[0].isupper():
                    correcao = correcao.capitalize()
                palavras_corrigidas.append(correcao)

```

```

        else:
            palavras_corrigidas.append(p)

    espacos = re.match(r'^\s*', linha).group(0)
    linha_corrigida = espacos + ' '.join(palavras_corrigidas)
    linhas_corrigidas.append(linha_corrigida)

    return '\n'.join(linhas_corrigidas)

def ocr_pdf_para_texto(pdf_path):
    """
    Extrai texto de todas as páginas do PDF, aplica OCR quando
    necessário
    e corrige erros ortográficos preservando parágrafos.
    """
    pdf = fitz.open(pdf_path)
    texto_completo = ""

    for i, pagina in enumerate(pdf):
        numero = i + 1
        print(f"Processando página {numero}...")

        # Extrai texto direto do PDF
        texto = pagina.get_text("text")

        # Se a página tiver pouco texto, aplica OCR
        if len(texto.strip()) < 600:
            pix = pagina.get_pixmap(dpi=250)
            img_path = os.path.join(TMPDIR, f"pagina {numero}.png")
            pix.save(img_path)
            texto = extrair_texto_tesseract(img_path)
            os.remove(img_path)

        texto_completo += f"\n\n--- Página {numero} ---\n\n{texto}"

    pdf.close()

    # Corrige automaticamente erros de OCR e ortografia
    # texto_corrigido = corrigir_texto_ocr(texto_completo)
    texto_corrigido = corrigir_texto_ocr(texto_completo)
    texto_corrigido = corrigir_frases(texto_corrigido)

    return texto_corrigido.strip()

```

Explicação:

```
CACHE = {}
```

- Cria um dicionário vazio para armazenar resultados temporários de correções.

```
DICIONARIO_URL = "https://www.ime.usp.br/~pf/dicios/br-utf8.txt"
print("♦ Baixando dicionário...")
r = requests.get(DICIONARIO_URL)
```

```
DICIONARIO_BASE = set(line.strip().lower() for line in
r.text.splitlines() if line.strip())
```

- Define a URL do dicionário oficial do IME-USP (contém milhares de palavras do português).
- Faz o download via requests.get().
- Limpa cada linha (removendo espaços e vazios), coloca tudo em minúsculas e converte em um set (conjunto) para busca rápida.
- Motivo técnico: o uso de set() torna a verificação de palavras O(1) (muito mais rápida).
- Isso é essencial para comparar milhares de palavras OCR em tempo real.

```
DICIONARIO_TECNICO = {
    "siggeo", "sig", "erpotech", "geotiff", "cad/gis", "qgis",
    "geoprocessamento", "geodésia", "geotecnologias",
    "georreferenciamento",
    "cartografia", "geoespacial", "teledetecção", "altimetria",
    "hidrografia", "topografia", "geocodificação",
    "georreferenciadas", "avanços", "divisão", "instalações",
    "unidade", "Seção", "empresas", "aculturamento"
}
```

- Conjunto de termos específicos da área de geoprocessamento e cartografia.

```
DICIONARIO = DICIONARIO_BASE.union(DICIONARIO_TECNICO)
```

```
print(f"[DICIONÁRIO] {len(DICIONARIO):,} palavras carregadas (incluindo técnicas).")
```

- Une o dicionário base e o técnico num só.

```
TERMOS_ESPECIAIS = {"SIGeo", "ERPTech", "GeoTIFF", "CAD/GIS", "QGIS"}
TERMOS_ESPECIAIS_SEM_ACENTO = set(unidecode(t.lower()) for t in
TERMOS_ESPECIAIS)
```

```
TERMOS_APRENDIDOS = set()
```

- Define siglas e nomes próprios que devem sempre ser preservados.
- Gera uma versão sem acento (unidecode) para comparação robusta.
- TERMOS_APRENDIDOS guarda palavras que o sistema corrige dinamicamente com o tempo.

As estruturas CORRECOES_FIXAS e CORRECOES_FRASES são mapas de substituição direta, criadas a partir da observação de erros recorrentes do Tesseract.

```
def normalizar_hifen_linhas(texto):  
    """  
    Junta palavras quebradas por hífen no final da linha, preservando  
    indentação e bullets.  
    """  
    linhas = texto.splitlines()  
    linhas_corrigidas = []  
    i = 0  
    while i < len(linhas):  
        linha = linhas[i]  
        if linha.rstrip().endswith('-') and i + 1 < len(linhas):  
            prox = linhas[i + 1].lstrip()  
            linha = linha.rstrip()[:-1] + prox  
            i += 1  
        linhas_corrigidas.append(linha)  
        i += 1  
  
    return '\n'.join(linhas_corrigidas)
```

- juntar palavras que foram separadas por hífen no final de uma linha (hifenização de quebra de linha)

```
CORRECOES_FIXAS_EXPANDIDO = {}  
for k, v in CORRECOES_FIXAS.items():  
    CORRECOES_FIXAS_EXPANDIDO[k] = v  
    sem_acento = unidecode(k)  
    if sem_acento != k:  
        CORRECOES_FIXAS_EXPANDIDO[sem_acento] = v
```

```
CORRECOES_FIXAS = CORRECOES_FIXAS_EXPANDIDO
```

CORRECOES_FIXAS_EXPANDIDO:

- cria novo dicionário vazio.

```
for k, v in CORRECOES_FIXAS.items():
```

- itera pares (errado → certo).

`CORRECOES_FIXAS_EXPANDIDO[k] = v`

- adiciona a correção original.

`sem_acento = unidecode(k)`

- obtém versão sem acento de k (ex.: "Seção" → "Secao").

`if sem_acento != k:`

- só adiciona a versão sem acento se ela for diferente (evita redundância).

`CORRECOES_FIXAS_EXPANDIDO[sem_acento] = v`

- adiciona a chave sem acento apontando para mesma correção.

`CORRECOES_FIXAS = CORRECOES_FIXAS_EXPANDIDO`

- troca o dicionário original pela versão expandida.

```
def distancia_levenshtein(a, b):
    if a == b: return 0
    if len(a) == 0: return len(b)
    if len(b) == 0: return len(a)
    matrix = [[0]*(len(b)+1) for _ in range(len(a)+1)]
    for i in range(len(a)+1): matrix[i][0] = i
    for j in range(len(b)+1): matrix[0][j] = j
    for i in range(1, len(a)+1):
        for j in range(1, len(b)+1):
            custo = 0 if a[i-1] == b[j-1] else 1
            matrix[i][j] = min(matrix[i-1][j]+1, matrix[i][j-1]+1,
                                matrix[i-1][j-1]+custo)

    return matrix[len(a)][len(b)]
```

Tratos iniciais (checar igualdade ou strings vazias) para retornar resultados triviais e evitar computação desnecessária.

```
matrix = [[0]*(len(b)+1) for _ in range(len(a)+1)]
```

- cria tabela (m+1)×(n+1).

Inicializa primeira coluna/linha com índices (custo de transformar string vazia em prefixos).

Loop duplo preenche a matriz usando relação de recorrência:

```
custo = 0 if a[i-1] == b[j-1] else 1
```

- custo de substituição.

```
matrix[i][j] = min(...)
```

- escolha entre inserção, deleção ou substituição.

Uso prático: ajuda a decidir se uma palavra OCRizada é próxima de uma palavra do dicionário. Em seu código você usa `get_close_matches` (difflib) que já oferece heurística, mas ter Levenshtein implementado permite criar regras customizadas (ex.: aceitar correções só se distância ≤ 2).

```
def aplicar_acentos(original, corrigida):
    resultado = ""
    for o, c in zip(original, corrigida):
        if re.match(r'[áéíóúãõâêôàèìòùç]', o.lower()):
            resultado += o
        else:
            resultado += c
        resultado += corrigida[len(resultado):]

    return resultado
```

- preserva acentuação caso esteja presente na palavra original extraída pelo OCR quando apropriado — útil se a palavra corrigida perdeu acento que o OCR havia recuperado corretamente em alguns caracteres.

```
def corrigir_frases(texto):
    for errado, certo in CORRECOES_FRASES.items():
        if errado in texto:
            texto = texto.replace(errado, certo)
            print(f"♦ Frase corrigida: '{errado}' -> '{certo}'")

    return texto
```

- aplica substituições completas de trechos/frases que o OCR costuma gerar erradamente, como casos fixos onde a correção depende de contexto maior que palavra por palavra.

```
def corrigir_texto_ocr(texto):
    """
```

```

    Corrigir texto OCR mantendo:
    - Indentação e bullets
    - Quebra de linhas
    - Hífens no final de linha
    - Siglas e termos técnicos
    """

    texto = normalizar_hifen_linhas(texto)

    matches = tool.check(texto)
    texto = language_tool_python.utils.correct(texto, matches)

    for errado, certo in CORRECOES_FIXAS.items():
        texto = re.sub(rf'\b{re.escape(errado)}\b', certo, texto)

    for errado, certo in CORRECOES_FRASES.items():
        texto = texto.replace(errado, certo)

    linhas = texto.splitlines()
    linhas_corrigidas = []

    for linha in linhas:
        palavras = re.findall(r'\b[\w\']+\b', linha)
        palavras_corrigidas = []

        for p in palavras:
            palavra_lower = p.lower()
            palavra_sem_acento = unidecode(palavra_lower)

            if len(p) <= 2 or p.isupper() or p.isdigit() or '-' in p:
                palavras_corrigidas.append(p)
                continue

            if p in TERMOS_ESPECIAIS or palavra_sem_acento in
TERMOS_ESPECIAIS SEM ACENTO:
                palavras_corrigidas.append(p)
                continue

            if palavra_lower in DICIONARIO:
                palavras_corrigidas.append(p)
                continue

            match_sem_acento = next((w for w in DICIONARIO if
unidecode(w) == palavra_sem_acento), None)
            if match_sem_acento:
                correcao = match_sem_acento
                if p[0].isupper():
                    correcao = correcao.capitalize()
                palavras_corrigidas.append(correcao)
            continue

```

```

        sugestoes = get_close_matches(palavra_sem_acento,
DICCIONARIO, n=1, cutoff=0.9)
        if sugestoes:
            correcao = sugestoes[0]
            if p[0].isupper():
                correcao = correcao.capitalize()
            palavras_corrigidas.append(correcao)
        else:
            palavras_corrigidas.append(p)

    espacos = re.match(r'^\s*', linha).group(0)
    linha_corrigida = espacos + ' '.join(palavras_corrigidas)
    linhas_corrigidas.append(linha_corrigida)

return '\n'.join(linhas_corrigidas)

texto = normalizar_hifen_linhas(texto)

```

- Hifens quebram palavras que mudariam a tokenização. Se juntarmos depois da correção gramatical, a LanguageTool ou substituições poderiam tratar a palavra como duas separadas, perdendo contexto. Portanto normalizar hífens antes garante tokens corretos.

```
matches = tool.check(texto)
```

```
texto = language_tool_python.utils.correct(texto, matches)
```

Executa LanguageTool (correção gramatical/ortográfica contextual). Faz isso antes das correções fixas e do dicionário por dois motivos:

- LanguageTool consegue reparar problemas sintáticos complexos (concordância, preposição, dupla palavra), melhorando a estrutura geral.
- Aplicar LanguageTool antes evita que substituições fixas se conflitem com sugestões gramaticais que dependem de contexto de frase.

```
for errado, certo in CORRECOES_FIXAS.items():
```

```
    texto = re.sub(rf'\b{re.escape(errado)}\b', certo, texto)
```

- Aplica substituições diretas por palavra (usa \b para marcar limites de palavra e re.escape para evitar que caracteres especiais causem regex inválida).

- Essas correções pegam casos persistentes que nem sempre LanguageTool corrige (erros típicos do OCR).

```
for errado, certo in CORRECOES_FRASES.items():
```

```
    texto = texto.replace(errado, certo)
```

- Aplica correções de frase inteira (podem ter caracteres que não se encaixam em \b), consertando trechos inteiros.

```
linhas = texto.splitlines()
```

```
linhas_corrigidas = []
```

- Agora que a estrutura global está mais estável, vamos tratar palavra a palavra preservando linhas e formatação.

```
for linha in linhas:
```

```
    palavras = re.findall(r'\b[\'w\']-]+\b', linha)
```

```
    palavras_corrigidas = []
```

- `re.findall(r'\b[\'w\']-]+\b', linha)` extrai tokens (palavras, permitindo apóstrofes e hífen internos). Essa regex ignora pontuações externas (vírgulas, pontos).
- Construímos lista de palavras corrigidas por linha.

```
for p in palavras:
```

```
    palavra_lower = p.lower()
```

```
    palavra_sem_acento = unidecode(palavra_lower)
```

```
    if len(p) <= 2 or p.isupper() or p.isdigit() or '-' in p:  
        palavras_corrigidas.append(p)
```

```
    continue
```

Pré-filtros — preserva:

- palavras muito curtas (≤ 2): artigos/preposições, risco de “correção equivocada”;
- palavras totalmente em maiúsculas (siglas → não as altera);
- tokens numéricos (IDs, datas);
- palavras com - (hifenadas ou códigos) — evita corrigi-las automaticamente;

- Esses filtros reduzem falsos positivos.

```
if p in TERMOS_ESPECIAIS or palavra_sem_acento in TERMOS_ESPECIAIS_SEM_ACENTO:  
    palavras_corrigidas.append(p)  
  
    continue
```

- Preserva termos especiais (lista de siglas/nomes técnicos), comparando também versão sem acento (por segurança).

```
if palavra_lower in DICIONARIO:  
    palavras_corrigidas.append(p)  
  
    continue
```

- Se a palavra já existe no dicionário (minúscula), mantemos como está — confiança alta.

```
match_sem_acento = next((w for w in DICIONARIO if unidecode(w) == palavra_sem_acento), None)  
if match_sem_acento:  
    correcao = match_sem_acento  
    if p[0].isupper():  
        correcao = correcao.capitalize()  
    palavras_corrigidas.append(correcao)  
  
    continue
```

- Tenta encontrar uma palavra no dicionário cuja forma sem acento seja igual à token OCRizada sem acento.
- Isso recupera acentos perdidos de forma confiável.

```
sugestoes = get_close_matches(palavra_sem_acento, DICIONARIO, n=1, cutoff=0.9)  
if sugestoes:  
    correcao = sugestoes[0]  
    if p[0].isupper():  
        correcao = correcao.capitalize()  
    palavras_corrigidas.append(correcao)  
else:  
  
    palavras_corrigidas.append(p)
```

- Quando não há correspondência sem acento, usa `diffib.get_close_matches` para obter a palavra mais próxima (cutoff 0.9 = alta similaridade exigida).
- Se encontrar sugestão, aplica (ajustando capitalização), caso contrário mantém token original (preferível a uma "correção errada").

```
espacos = re.match(r'^\s*', linha).group(0)
linha_corrigida = espacos + ' '.join(palavras_corrigidas)
```

```
linhas_corrigidas.append(linha_corrigida)
```

- Recupera os espaços iniciais da linha (`^\s*`) para preservar indentação (importante para bullets, níveis de tópicos).
- Rejunta as palavras com um único espaço (normaliza múltiplos espaços internos).
- Acumula a linha corrigida.

```
return '\n'.join(linhas_corrigidas)
```

- Reconstrói o texto final com quebras de linha preservadas.

```
def ocr_pdf_para_texto(pdf_path):
    """
    Extrai texto de todas as páginas do PDF, aplica OCR quando
    necessário
    e corrige erros ortográficos preservando parágrafos.
    """
    pdf = fitz.open(pdf_path)
    texto_completo = ""

    for i, pagina in enumerate(pdf):
        numero = i + 1
        print(f"Processando página {numero}...")

        # Extrai texto direto do PDF
        texto = pagina.get_text("text")

        # Se a página tiver pouco texto, aplica OCR
        if len(texto.strip()) < 600:
            pix = pagina.get_pixmap(dpi=250)
            img_path = os.path.join(TMPDIR, f"pagina_{numero}.png")
            pix.save(img_path)
            texto = extrair_texto_tesseract(img_path)
            os.remove(img_path)
```

```

    texto_completo += f"\n\n--- Página {numero} ---\n\n{texto}"

pdf.close()

# Corrige automaticamente erros de OCR e ortografia
# texto corrigido = corrigir_texto_ocr(texto_completo)
texto_corrigido = corrigir_texto_ocr(texto_completo)
texto_corrigido = corrigir_frases(texto_corrigido)

return texto_corrigido.strip()

pdf = fitz.open(pdf_path)

```

- Abre o PDF usando PyMuPDF (fitz).
- pdf passa a ser um objeto representando o documento; permite iterar páginas, extrair texto nativo ou renderizar a página como imagem.

```

texto_completo = ""

```

- Inicializa a string que irá acumular o conteúdo processado de todas as páginas. Usar uma string é simples e direto; para documentos muito grandes poderia usar list + ".join()" ao final por eficiência, embora aqui você já concatene por páginas (a diferença é pequena).

```

for i, pagina in enumerate(pdf):
    numero = i + 1

```

```

    print(f"Processando página {numero}...")

```

- Loop que percorre cada página do PDF.
- enumerate(pdf) fornece índice i (0-based) e o objeto pagina.
- numero = i + 1 deixa a numeração humana (1-based).
- print(...) é log/feedback simples para acompanhar progresso no terminal — útil ao processar lotes longos.

```

texto = pagina.get_text("text")

```

- Tenta extrair texto nativo da página (camada de texto do PDF).
- Quando o PDF foi gerado digitalmente (não escaneado), get_text("text") geralmente retorna texto perfeito, com ordenação correta e sem necessidade de OCR.

- Isso é muito mais rápido e preciso que aplicar OCR em tudo.

```
if len(texto.strip()) < 600:
```

Heurística: se o texto extraído for curto (menos de 600 caracteres), assume-se que a página provavelmente é uma imagem escaneada ou tem pouco texto nativo (por exemplo uma figura, uma tabela como imagem).

`texto.strip()`:

- remove espaços em branco; o comprimento real indica presença de conteúdo.

```
pix = pagina.get_pixmap(dpi=250)
```

- Renderiza a página como imagem (Pixmap) com 250 DPI.
- 250 DPI é um bom trade-off: aumenta legibilidade para OCR sem explodir o tamanho do arquivo; valores típicos para OCR ficam entre 200–300 DPI.

```
img_path = os.path.join(TMPDIR, f"pagina_{numero}.png")
```

- Monta o caminho do arquivo temporário onde a imagem será salva.
- TMPDIR deve ser uma pasta temporária existente (definida globalmente).
- Usar `os.path.join` garante compatibilidade de caminho entre sistemas operacionais.

```
pix.save(img_path)
```

- Salva a imagem (PNG) no disco. O Tesseract normalmente consome arquivos ou objetos PIL, então salvar temporariamente é prático.

```
texto = extrair_texto_tesseract(img_path)
```

- Chama a função `extrair_texto_tesseract`, que pré-processa a imagem (filtros, CLAHE, binarização) e executa `pytesseract.image_to_string`.
- Resultado é a string OCR para a página renderizada.

```
os.remove(img_path)
```

- Remove o arquivo temporário para não acumular lixo em disco. Boa prática de limpeza.

```
texto_completo += f"\n\n--- Página {numero} ---\n\n{texto}"
```

- Concatena ao texto_completo um separador visual por página e o texto da página.
- A marcação --- Página X --- facilita debugging, revisões manuais e mapeamento do texto de volta à página original.

```
pdf.close()
```

- Fecha o documento PDF liberando recursos. Boa prática fechar handle de arquivo.

```
texto_corrigido = corrigir_texto_ocr(texto_completo)
```

```
texto_corrigido = corrigir_frases(texto_corrigido)
```

Pós-processamento em duas etapas:

- corrigir_texto_ocr(texto_completo) — aplica normalização de hífens, LanguageTool, correções fixas, dicionário e matching palavra-a-palavra (o motor de correção que discutimos).
- corrigir_frases(texto_corrigido) — aplica substituições de frases inteiras (CORRECOES_FRASES) e imprime logs.

```
return texto_corrigido.strip()
```

- Retorna o texto final, com strip() removendo espaços/linhas extras nas extremidades.
- Resultado pronto para armazenamento, indexação ou exportação.

Dessa forma o resultado vem de forma correta:

```
PS C:\xampp\htdocs\leis_externo> python
C:\xampp\htdocs\leis_externo\processamento_OCR.py
>>
♦ Baixando dicionário...
[DICIONÁRIO] 261,495 palavras carregadas (incluindo técnicas).
```

♦ Processando lote a partir do ID 21312...
Processando página 1...
Processando página 2...
Processando página 3...
Processando página 4...
♦ Frase corrigida: 'Profissão eximias' -> 'Profissão exigida'
♦ Frase corrigida: 'suplementavas se necessário' -> 'suplementadas se necessário'
✅ Lei 21312 atualizada
🏁 Lote ID 21312 finalizado!

🏆 Todos os lotes processados!

Resultado:

Página 1

Fonte Departamento de Relações Administrativas Prefeitura de Guarulhos

DECRETO n ° 19542

de 27 de agosto de 1996

Dispõe Sobre Define atribuições de sub-unidades
criadas pelo Decreto n ° 19 533 96

O BACHAREL VICENTINO PAPOTITO PREFEITO DE
GUARULHOS no uso de suas atribuições legais
Considerando que compete a Secretaria de Economia e Planejamento
a definição de metodologia que garantam o planejamento e gerenciamento
do Município
para adequar suas aptidões sempre em sintonia com os mais modernos
recursos
tecnológicos
Considerando a necessidade de modernização das rotinas de
procedimentos técnicos-administrativos voltados a otimização quanto a
prestação de
serviços
Considerando os investimentos já implementados e dirigidos as
primeiras fases de implantação do projeto SIGeo
Considerando a realidade quanto a utilização desta tecnologia junto as
várias unidades administrativas desta Prefeitura já incorporados as
suas rotinas de
trabalho
Considerando a existência de produtos já gerados e implantados pela
Comissão Técnica Multidisciplinar formada por técnicos desta Secretaria
Considerando o aprimoramento técnico desta equipe detendo

conhecimentos específicos nas áreas envolvidas
Considerando a existência de um parque tecnológico de equipamentos de última geração voltados as áreas de informática e geotecnologias geoprocessamento
geodésia cartografia topografia entre outros
Considerando a necessidade de manter continuamente atualizada a base cartográfica do município assim como também tornar compatível e confiável seu banco de dados gráfico digital e tabular entre outros
Considerando a necessidade de garantir a institucionalização da estrutura do projeto SIGeo visando assegurar a continuidade deste de forma que sejam implantadas e desenvolvidas todas as etapas previstas e
Considerando o artigo 29 da Lei n ° 8 214 91 e o que consta do processo n ° 17 092 96

DECRETA

Artigo 1º Ficam definidas as atribuições específicas das subunidades da Secretaria de Economia e Planejamento criadas através do Decreto n ° 19 533 96 conforme segue

Página 2

Denominação da
unidade Divisão Técnica do SIGeo
Sistema de Informações Georreferenciadas

Condição de provimento
Nível Universitário com especializa
caso em Geoprocessamento ou reconhecida
experiência na área

Profissão exigida
Engenheiro Geólogo Geógrafo ou Arquiteto

Atribuições específicas
Gerenciar o Sistema Cartográfico Municipal SCM GUL
Normatizar e padronizar a execução de levantamentos
topográficos cartográficos e geodésicos do Município

Gerir a implantação e desenvolvimento do Sistema de Informação Georreferenciadas de Guarulhos SIGeo-Guarulhos

Promover o aprimoramento técnico-científico da equipe multidisciplinar responsável pela condução do Sigeo-Guarulhos visando o desenvolvimento de todas as etapas do Projeto de forma adequada e atrelada aos avanços tecnológicos disponíveis no mercado x

Estabelecer as diretrizes para a determinação das instalações e

equipamentos adequados a todas as etapas do Projeto

Definir os sistemas aplicados ao geoprocessamento

Divulgar o SIGeo junto as empresas privadas ou publicas
prestaras de serviços prevendo o intercâmbio de informações de
interesse a administração municipal

Elaborar as especificações técnicas dos Termos de Referência
Visando a contratação de serviços na área de topografia
cartografia geodésia e geoprocessamento em projetos correlatos
Bem como acompanhar os serviços e ou projetos contratados
visando o seu controle de qualidade e
Arbitrar e propor parecer sobre questões fronteiras Quanto
aos limites do território do Município de Guarulhos

Denominação da
unidade Seção Técnica de Geoprocessamento

Condição de provimento Profissão exigida

Nível Universitário com especializa Engenheiro Geólogo Geocao em
Geoprocessamento ou reconhecida geógrafo ou Arquiteto
experiência na área

Atribuições específicas

Levantar o fluxo de informações entre as unidades administrai
administrativas
Promover a concepção e desenvolvimento de sistemas compatíveis
com as necessidades técnico-administrativas
Disseminar o uso da tecnologia de Geoprocessamento junto as
unidades administrativas Prevendo o aculturamento treinamento
e acompanhamento l

Fonte Departamento de Relações Administrativas Prefeitura de Guarulhos

Página 3

Estabelecer o estruturar as rotinas de manutenção das bases doe
dados gráficos e alfanuméricos

Gerenciar a ligação e compatibilizava das bases de dados gráficos
gráficos e alfanuméricos

Promover a geração de novas informações pelo cruzamento de da-r
dos existentes abrangendo avaliações espaciais e temporada
a fim de alimentar o Sistemas

Gerir e manter a operacionalidade do Sistema e

Promover a aquisição de informações alfanuméricas sociais demográficas econômicas e etc das mais variadas fontes junto as instituições provedorias de tais dados para o posterior tratamento e compatibilizava a fim de carregá-las no Sistema Denominação da

unidade Seção Técnica de Geociências

Condição de provimento Profissão exigida

Nível Universitário com especializa Engenheiro Geólogo Geógrafo cã em Geoprocessamento ou reconhecida grafo ou Arquiteto experiência na área

Atribuições específicas

Manter a base cartográfica digital do Município continuamente atualizadas

Zelar pela guarda Controle e manutenção de todo acervo cartográfico gerado a partir do mapeamento digital do município bem como o acervo aerofotográfico a ele relacionados

Promover a manutenção da rede básica de apoio geodésico

Adensar a rede básica de apoio geodésico a fim de prover novas áreas ou melhorar as condições de realização dos trabalhos topográficos

Fornecer e dar atendimento as solicitações de referências de amarração para a realização de levantamentos topográficos Na forma de monografias de marcos da rede básica de apoio geodésico z Analisar os produtos gerados em trabalhos das áreas de topografia Cartografia e geodésia realizados no Município para poste rio migração para a base cartográfica oficial x

Promover a manutenção sistemática e guarda dos equipamentos empregados na manutenção e densificação da rede básica de apoio geodésicos

Produzir através de edição recorte e função de plantas da base cartográfica do Município plantas em outras escalas e ou formatos tos específicos E

Elaborar através da conversão de mapas temáticos existentes Mapas no formato digital para carga e alimentação do Sistema

Artigo 2º As despesas decorrentes do presente Decreto correrão por conta de dotações próprias do orçamento vigente suplementadas se necessário

Artigo 3º Este decreto entra em vigor na data de sua publicação
revogadas as disposições em contrário

Fonte Departamento de Relações Administrativas Prefeitura de Guarulhos

Página 4

Fonte Departamento de Relações Administrativas Prefeitura de Guarulhos

Guarulhos 27 de agosto de 1996

BEL VICENTINO PAPOTITO
Prefeito

Dr NEWTON DE OLIVEIRA VANS
Secretário de Administração

Registrado no Departamento de Relações Administrativas Secretaria do
Governo
Municipal da Prefeitura Municipal de Guarulhos e afixado no lugar
público de costume aos
vinte e sete dias do mês de agosto de mil novecentos e noventa e seis

DEISE RAMPADO
Diretora

Publicado no Jornal Folha Metropolitana em 28 de agosto de 1996

REVOGADO PELO DECRETO n ° 22572 2004